التصميم الكائني باستخدام UML

مقدمة أساسية: ما هو OO Design؟

- **نظام OOP:** مجموعة كائنات متفاعلة تحتفظ كل منها بـ حالة خاصة (**State**) وتوفر عمليات على تلك الحالة — الحالة خاصة (**Private**) ولا يمكن الوصول إليها مباشرة من الخارج.
- **عملية OO Design:** تعني تصميم Class Objects والعلاقات بينها — وعند التنفيذ تُنشأ الكائنات ديناميكياً من تعريفات الكلاسات.

الخطوات الخمس لعملية OO Design

1. **Interactions & System Context**
2. **Architectural Design**
3. **Object Class Identification**
4. **Design Models**
5. **Interface Specification**

التصميم ليس عملية خطية صارمة — بل تطوير متكرر بالتراجع والتقدم حسب ما يتاح من معلومات.

---

1 — الخطوة الأولى: Interactions & System Context

الهدف: فهم العلاقة بين النظام وبيئته الخارجية — وهو ما يُحدد حدود النظام (**System Boundaries**).

يُستخدم نوعان مكمّلان من النماذج:

النموذج	النوع	الوصف
System Context Model	هيكلي (Structural)	يعرض الأنظمة الأخرى في محيط النظام المصمَّم.
Interaction Model	ديناميكي (Dynamic)	يعرض كيف يتفاعل النظام مع بيئته أثناء الاستخدام.

- **مثال محطة الطقس (Weather Station):**
  - **Context Model:** يُظهر ارتباط المحطة بـ: Weather Information System + Satellite + Control System.
  - **Use Case Model (Interaction):** 7 حالات استخدام ← Report weather, Report status, Restart, Shutdown, Reconfigure, Power Save, Remote control.

---

2 — الخطوة الثانية: Architectural Design

بعد تعريف السياق ← تُصمَّم المعمارية العليا للنظام عبر تحديد المكونات الرئيسية وعلاقاتها، باستخدام نمط معماري مثل **Layered** أو **Client-Server**.

- **مثال محطة الطقس — المعمارية العليا:**
  - **النمط المستخدم:** Listener Model — كل Subsystem يستمع للرسائل ويأخذ ما يخصه.
  - **الميزة:** لا يحتاج المُرسِل أن يعرف عنوان المُستقبِل ← سهولة تغيير الـ configuration.

---

3 — الخطوة الثالثة: Object Class Identification

الهدف: اكتشاف الكائنات الأساسية في النظام.

3 طرق لتحديد الكائنات:

1. **Grammatical Analysis:** الأسماء = Objects + Attributes / الأفعال = Operations (Abbott 1983).
2. **Tangible Entities:** الأشياء الملموسة، الأدوار، الأحداث، الوحدات (Wirfs-Brock 1990).
3. **Scenario-Based Analysis:** تحليل سيناريوهات الاستخدام لتحديد الكائنات والعمليات (Cunningham 1989 & Beck).

⚠ **في الواقع:** تستخدم مصادر متعددة معاً — وصف النظام + معرفة المجال + تحليل السيناريو.

- **مثال محطة الطقس — 5 كائنات محددة:**
  - **Weather Station (System Object):** الواجهة الرئيسية مع البيئة الخارجية.
  - **Weather Data (System Object):** معالجة وإرسال ملخص بيانات الطقس.
  - **Ground Thermometer (Domain Object):** قياس درجة حرارة الأرض.
  - **Anemometer (Domain Object):** قياس سرعة واتجاه الرياح.
  - **Barometer (Domain Object):** قياس الضغط الجوي.

---

4 — الخطوة الرابعة: Design Models

النماذج هي الجسر بين المتطلبات والتطبيق — يجب أن تكون مجردة بما يكفي ولكن بـ تفاصيل كافية للمبرمجين.

نوعا نماذج UML الأساسيان:

النوع	الوصف	الأمثلة
Structural Models (ثابتة)	تصف البنية الساكنة للنظام.	Class Diagrams, Subsystem Models.
Dynamic Models (ديناميكية)	تصف التفاعلات في وقت التشغيل.	Sequence Diagrams, State Diagrams.

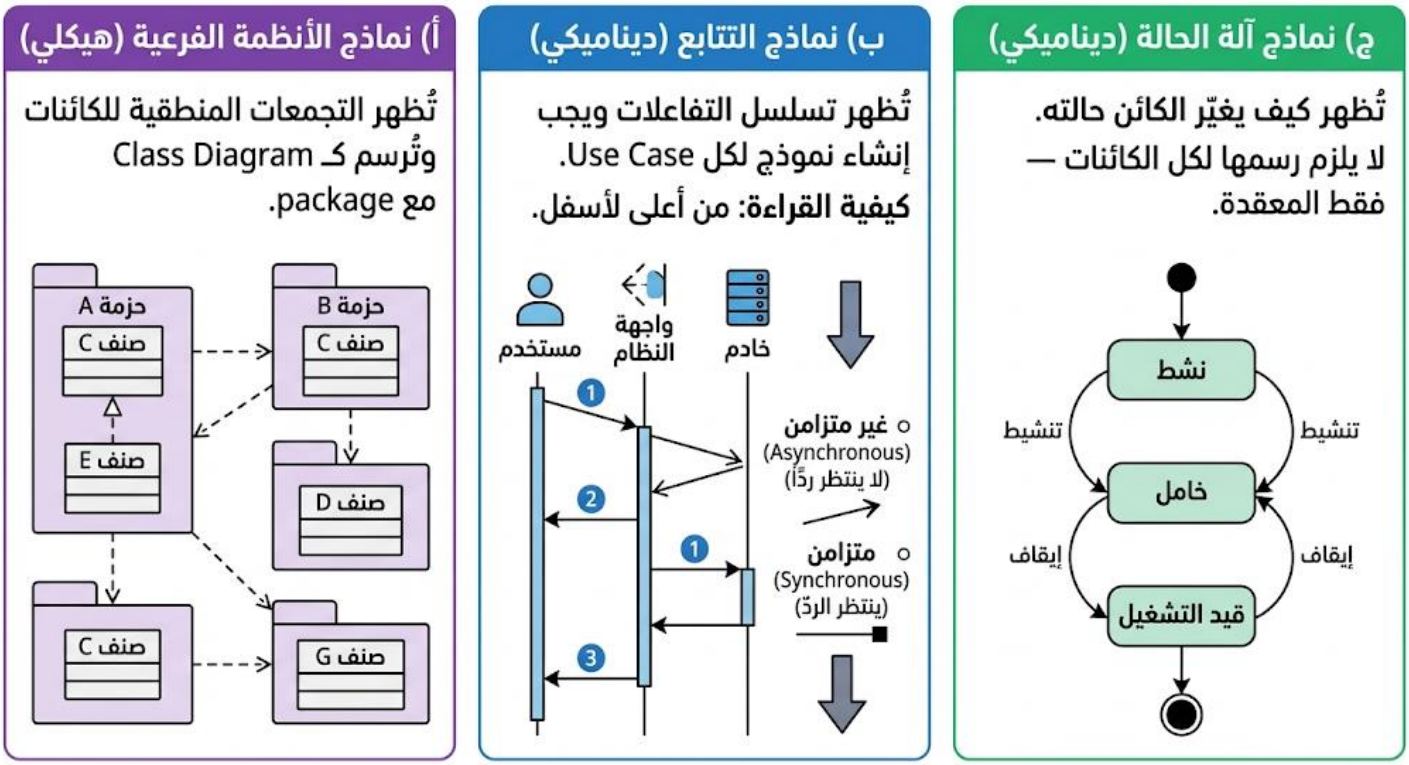
أهم 3 أنواع نماذج UML في التصميم:

- أ) **Subsystem Models (هيكلي):** تُظهر التجمعات المنطقية للكائنات وتُرسم كـ Class Diagram مع packages.
- ب) **Sequence Models (ديناميكي):** تُظهر تسلسل التفاعلات ويجب إنشاء نموذج لكل Use Case.

كيفية القراءة: من أعلى لأسفل.

- سهم مائل (stick arrowhead) = **Asynchronous** (لا ينتظر ردّاً).
- سهم مستقيم (squared arrowhead) = **Synchronous** (ينتظر الردّ).

- **ج) State Machine Models (ديناميكي):** تُظهر كيف يغيّر الكائن حالته. لا يلزم رسمها لكل الكائنات — فقط المعقدة.



---

5 — الخطوة الخامسة: Interface Specification

الهدف: تحديد الواجهات (Interfaces) حتى يمكن تطوير المكونات بالتوازي.

قواعد Interface Specification:

- لا تُدرج تفاصيل تمثيل البيانات (Attributes) داخل الـ Interface.
- أدرج العمليات للوصول إلى البيانات وتحديثها فقط.
- يُعرَّف في UML بنفس Class Diagram لكن بدون Attributes + الـ stereotype: **«interface»**.
- دلالات الـ Interface تُعرَّف بـ **OCL** (Object Constraint Language).

نقاط MCQ احفظها جيداً

- **OO Design** = تصميم Object Classes + العلاقات بينها.
- **الخطوات الخمس:** Context → Architecture → Object ID → Design Models → Interfaces.
- **Context Model** = هيكلي | **Interaction Model** = ديناميكي.
- **Use Case** = نموذج تفاعلي يصف كيف يتفاعل النظام مع بيئته.
- **3 طرق تحديد الكائنات:** Grammatical + Tangible Entities + Scenario-Based.
- **Grammatical Analysis:** الأسماء = Objects | الأفعال = Operations.
- **Structural Models** = ثابتة (Class, Subsystem) | **Dynamic Models** = متحركة (Sequence, State).
- UML يدعم 13 نوع من النماذج — لكن الأهم 3: **Subsystem + Sequence + State Machine**.
- **Sequence Diagram** يُقرأ من أعلى لأسفل.
- **Stick arrowhead** = Asynchronous (لا ينتظر ردّاً).
- **Squared arrowhead** = Synchronous (ينتظر الردّ).
- **Interface في UML** = Class Diagram بدون Attributes + stereotype **«interface»**.
- **OCL** = لغة تحديد دلالات الـ Interface.
- الـ Interface تُحسّن الـ **Maintainability** لأن تمثيل البيانات مخفي.
- كائن واحد يمكنه تطبيق واجهات متعددة.
- **Listener Model** = نمط معماري تستمع فيه كل subsystem للرسائل وتأخذ ما يخصها.

أنماط التصميم (Design Patterns)

Design Pattern = وصف مشكلة + جوهر حلّها بحيث يمكن إعادة استخدام الحل في سياقات مختلفة — ليس مواصفة تفصيلية بل قالب لحل مُجرَّب ومثبت.

Hillside Group: Patterns are ways to describe best practices, good designs, and capture experience in a way that it is possible for others to reuse this experience."

من أين جاء مفهوم Design Patterns؟

الشخص	السنة	الإسهام
Christopher Alexander	1979	اقترح فكرة الأنماط في تصميم المباني - الحل الناجح يُعاد استخدامه
Gang of Four (GoF) — Gamma, Helm, Johnson, Vlissides	1995	نقلوا الفكرة إلى هندسة البرمجيات في كتابهم الشهير
Siemens Group (Buschmann وآخرون)	1996-2007	نشروا مجموعة كتب أخرى عن أنماط البرمجيات

Gang of Four = أشهر مرجع لـ Design Patterns - وثّقوا 23 نمطاً للأغراض العامة.

---

1 العناصر الأربعة لأي Design Pattern (حسب Gang of Four)

هذه الأربعة عناصر إلزامية لكل نمط:

1. **Name:** اسم ذو معنى يُعرِّف النمط ويكون مرجعاً له في النقاشات.
2. **Problem Description:** يشرح متى يُطبَّق النمط — المشكلة وسياقها (motivation + applicability).
3. **Solution Description:** قالب للحل — ليس تصميماً محدداً بل علاقات + مسؤوليات المكونات + يُعبَّر عنه غالباً بمخطط UML.
4. **Consequences:** نتائج وتوازنات تطبيق النمط — تساعد في قرار استخدامه من عدمه.

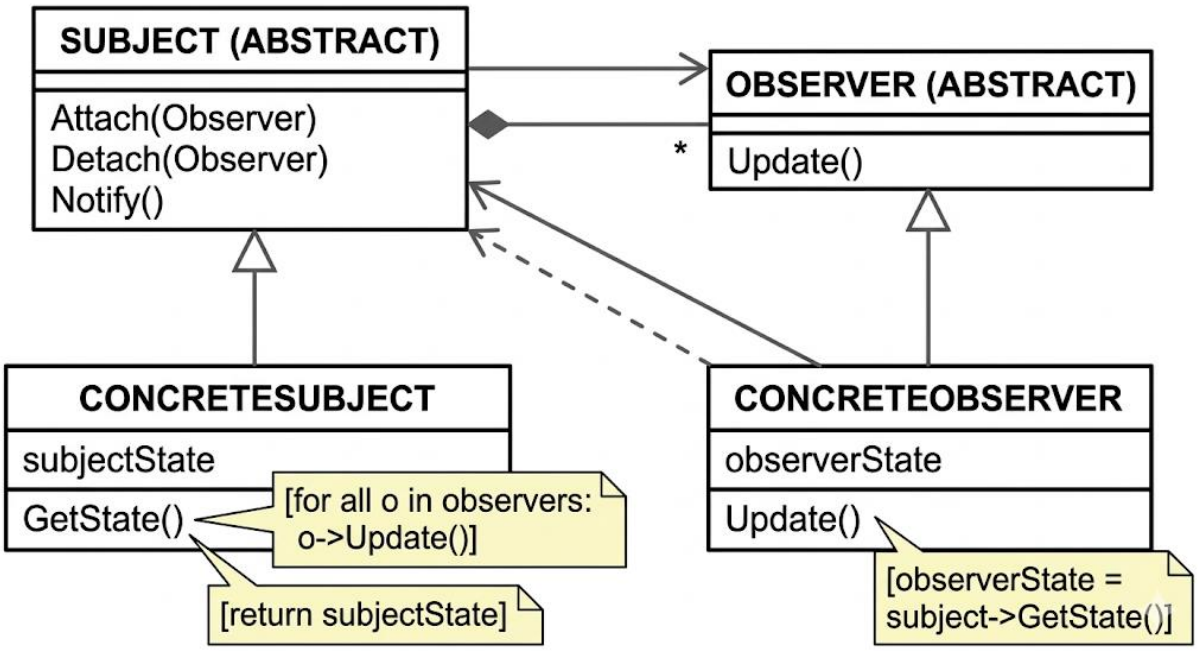
---

2 Observer Pattern — المثال التفصيلي

هذا النمط هو المثال الرئيسي في الكتاب ومن أهم أنماط:

العنصر	المحتوى
Pattern Name	Observer
Description	يفصل عرض حالة الكائن عن الكائن نفسه — عند تغيُّر الحالة يُخطَر جميع العروض تلقائياً
Problem	تحتاج إلى عروض متعددة لنفس البيانات (جدول + مخطط بياني) — لا تعرف جميعها مسبقاً — أي تغيير يجب أن يُحدِّث جميع العروض
When to use	عندما تحتاج أكثر من طريقة عرض لحالة كائن واحد، ولا يجب أن يعرف الكائن تفاصيل العروض
Solution	4 كائنات: Subject (مجرد) + Observer (مجرد) + ConcreteObserver + ConcreteSubject
Consequences	☑ اقتران ضعيف (minimal coupling) بين Subject والـ Observers

هيكل Observer Pattern بالـ UML:



---

3 أنماط Gang of Four الأربعة المذكورة في الكتاب

اسم النمط	المشكلة التي يحلها	الحل باختصار
Observer	إخبار عدة كائنات بتغيُّر حالة كائن آخر	فصل الـ Subject عن الـ Observers مع آلية Notify تلقائية
Façade	تنظيف الواجهات لعدة كائنات ذات صلة طُوِّرت تدريجياً	توفير واجهة موحدة بسيطة تختصر الواجهات المعقدة
Iterator	توفير طريقة موحدة للوصول لعناصر مجموعة بغض النظر عن تطبيقها	كائن Iterator يُجرِّد عملية التكرار عبر المجموعة
Decorator	توسيع وظيفة كائن موجود وقت التشغيل دون تغيير كلاسه	تغليف الكائن الأصلي بكائن Decorator يضيف وظائف جديدة

---

4 Patterns كأداة لإعادة الاستخدام (Reuse)

الأنماط تدعم إعادة الاستخدام على مستوى المفاهيم (Concept Reuse) وليس على مستوى الكود فقط:

مستوى الـ Reuse	الشرح
Component Reuse	تُعيد استخدام كود جاهز ← مُقيَّد بقرارات التصميم الأصلية (Algorithms, Interfaces)
Pattern Reuse	تُعيد استخدام الفكرة والهيكل ← تُكيّف التطبيق بحرية لتناسب نظامك

- ✅ **ميزة Patterns:** تُعيد استخدام الأفكار مع حرية تكييف التطبيق.
- ❌ **قيد Component:** قد يتعارض مع متطلباتك وقد يُدخل كفاءة منخفضة.

---

5 كيف تستخدم Patterns في التصميم؟

الطريقة العملية:

صمِّم الحل أولاً ← اصطدم بالمشكلة ← تعرَّف على النمط المناسب ← طبِّق النمط

⚠ **تحذير مهم:** تطبيق الأنماط يحتاج خبرة في التصميم — المبرمج غير الخبير حتى لو قرأ كتب الأنماط سيجد صعوبة في معرفة متى يطبقها ومتى يبتكر حلاً خاصاً.

---

6 أهمية Design Patterns في هندسة البرمجيات

- أصبحت لغة مشتركة بين المصممين — تشرح تصميمك بمجرد ذكر اسم النمط.
- تعتمد على خصائص OOP: Inheritance و Polymorphism لتوفير العمومية.
- الـ 23 نمطاً في كتاب Gang of Four تحت التركيز — ما وراء ذلك مئات الأنماط الأخرى.
- المبدأ عام: يُطبَّق على أي نوع من تصميم البرمجيات ليس OOP فقط.

نقاط MCQ احفظها جيداً

- ◈ **Christopher Alexander (1979)** = أصل فكرة الأنماط من تصميم المباني.
- ◈ **Gang of Four** = Gamma, Helm, Johnson, Vlissides — كتابهم عام 1995.
- ◈ Gang of Four وثّقوا **23 نمطاً** للأغراض العامة.
- ◈ **4 عناصر لأي Design Pattern:** Name + Problem + Solution + Consequences.
- ◈ الـ **Solution** = قالب (Template) وليس تصميماً محدداً جاهزاً.
- ◈ **Observer Pattern** = فصل حالة الكائن عن عروضه المتعددة.
- ◈ Observer يستخدم **4 كائنات:** Subject + Observer (abstract) + ConcreteSubject + ConcreteObserver.
- ◈ نتيجة Observer: **اقتران ضعيف** بين Subject والـ Observers.
- ◈ **Façade** = تبسيط واجهات معقدة | **Iterator** = وصول موحد للمجموعات | **Decorator** = توسيع وظائف وقت التشغيل.
- ◈ **Pattern Reuse** أمرن من Component Reuse — تُعيد الفكرة وتُكيِّف التطبيق.
- ◈ الأنماط تحتاج خبرة لتطبيقها — المبتدئون يجدون صعوبة في تحديد متى يستخدمونها.
- ◈ الأنماط أصبحت **مفردات مشتركة** يتحدث بها المصممون عن التصميم.
- ◈ الأنماط مرتبطة بـ OOP لكنها قابلة للتطبيق على أي نوع تصميم.

مسائل التطبيق العملي (Implementation Issues)

مقدمة: ما الذي يغطيه هذا القسم؟

القسم يُركّز على 3 مواضيع حيوية في التطبيق العملي لا تُغطَّى في كتب البرمجة العادية:

الموضوع	الفكرة الجوهرية
Reuse	لا تبنِ من الصفر — أعِد استخدام ما هو موجود
Configuration Management	تحكّم في التغييرات والإصدارات
Host-Target Development	بيئة التطوير ≠ بيئة التشغيل

---

1 — إعادة استخدام البرمجيات: Software Reuse

نبذة تاريخية

- من الستينيات حتى التسعينيات: كل شيء يُبنى من الصفر (From Scratch).
- الضغط على التكلفة والوقت جعل هذا النهج غير قابل للاستمرار — خاصة في الأنظمة التجارية والإنترنت.
- الان إعادة الاستخدام هي النهج الافتراضي لمعظم أنواع التطوير: Systems Web Systems, Scientific Software, Embedded.

4 مستويات إعادة الاستخدام (للحفظ)

```
المستوى الأعلى (تجريد)
        ▲
        | 4. System Level
        | 3. Component Level
        | 2. Object Level
        | 1. Abstraction Level
        ▼
المستوى الأدنى (تجريد)
```

المستوى	ما يُعاد استخدامه	الأمثلة
Abstraction Level	المعرفة والأفكار — لا كود مباشر	Design Patterns + Architectural Patterns
Object Level	كائنات وطرق من مكتبة جاهزة	JavaMail library لمعالجة البريد الإلكتروني
Component Level	مجموعات من الكائنات تعمل معاً تحتاج تكييف	UI Framework يُوفّر event handling + display management
System Level	إعادة استخدام نظام كامل مع تهيئة أو تعديل	COTS (Commercial Off-The-Shelf) systems

✅ فوائد إعادة الاستخدام

- تطوير أسرع مع مخاطر أقل وتكلفة أقل.
- البرنامج المُعاد استخدامه أكثر موثوقية لأنه اختُبر في تطبيقات أخرى.

❌ تكاليف إعادة الاستخدام (4 تكاليف)

التكلفة	الشرح
وقت البحث والتقييم	البحث عن برنامج مناسب + اختباره في بيئتك
تكلفة الشراء	بعض الأنظمة الجاهزة (COTS) مرتفعة جداً
تكلفة التكييف	تعديل البرنامج ليلائم متطلباتك
تكلفة التكامل	دمج برمجيات من مصادر مختلفة قد تتعارض افتراضاتها

🔔 **القاعدة الذهبية:** إعادة الاستخدام يجب أن تكون أول ما تفكر فيه قبل البدء في أي مشروع — قبل التصميم التفصيلي.

---

2 — إدارة التهيئة: Configuration Management)

Configuration Management = العملية الشاملة لإدارة النظام البرمجي المتغيّر — هدفها دعم عملية التكامل حتى يتمكن كل المطورين من الوصول للكود والوثائق بشكل منظّم ومتحكّم.

لماذا هي ضرورية؟ (3 مشاكل تحلها)

1. **مشكلة التضارب:** مطوران يعدّلان نفس المكوّن ← أحدهما يُلغي عمل الآخر.
2. **مشكلة الإصدارات:** المطورون يعملون على نسخ قديمة ← يُعيدون عملاً منجزاً.
3. **مشكلة الأعطال:** إصدار جديد معطوب ← يجب العودة لإصدار سابق يعمل.

4 أنشطة Configuration Management الأساسية

النشاط	الوصف	أدوات مثال
Version Management	تتبُّع الإصدارات المختلفة للمكوّنات + منع التضارب بين المطورين	Git, Subversion (SVN)
System Integration	تحديد أي إصدارات تدخل في كل version للنظام + بناء النظام تلقائياً	GNU Build System
Problem Tracking	تسجيل الأخطاء + تتبّع من يعمل عليها + متى تُحلّ	Bugzilla, GitHub Issues
Release Management	التخطيط لإصدارات جديدة للعملاء + تنظيم توزيع البرنامج	GitHub Releases

تدفق Configuration Management:

```
Change Proposals ⟶ Change Management ⟶ System Versions
                                                ↓
Component Versions ⟶ Version Management ⟶ System Building
                                                ↓
                    Release Management ⟶ System Releases
```

💡 أدوات CM تُثبَّت عادةً داخل بيئة التطوير المتكاملة (IDE) مثل Eclipse.

3 — التطوير Host-Target

المفهوم الأساسي

Host-Target Model = نموذج التطوير الاحترافي حيث يجري التطوير على جهاز (Host) والتشغيل على جهاز آخر (Target).
Plaintext

```
┌──────────────────┐              ┌──────────────────┐
│ Development      │              │ Execution        │
│ Platform (Host)  │ ──────────→  │ Platform (Target │
│                  │  Download    │                  │
│ • IDE            │ Software     │ • Libraries      │
│ • Compilers      │              │ • Related Systems│
│ • Testing Tools  │              │ • Databases      │
└──────────────────┘              └──────────────────┘
```

- **Platform** = أكثر من مجرد Hardware.
- **Platform** = الأجهزة + نظام التشغيل + البرمجيات الداعمة (DBMS, middleware, IDE...).

متى يكون Host = Target ومتى يختلفان؟

الحالة	المثال	التعامل
Host = Target	تطوير Target — Java هو JVM	اختبار مباشر على نفس الجهاز
Host ≠ Target	Embedded Systems + Mobile Systems	تحميل الكود على الجهاز الهدف أو استخدام Simulator

Simulators — المحاكيات

تُستخدم خاصةً في Embedded Systems لمحاكاة الأجهزة (Sensors) والبيئة:

- **ميزتها:** كل مطور عنده بيئة تشغيل خاصة به ← تسريع التطوير بدون تحميل على الأجهزة الحقيقية.
- **عيبها:** مكلفة التطوير ← متاحة فقط لأشهر المعماريات.

6 أدوات يجب أن توفرها بيئة التطوير

الأداة	الغرض
Integrated Compiler + Syntax Editor	كتابة وتحرير وترجمة الكود
Language Debugger	تتبُّع الأخطاء
Graphical Editing Tools	محررات UML
Testing Tools	كـ JUnit لتشغيل الاختبارات تلقائياً
Visualization Tools & Refactoring	دعم التحسين
Configuration Management Tools	إدارة الإصدارات والبناء

Eclipse هو أشهر General-Purpose IDE — يعتمد على plug-in architecture يمكن تخصيصه لأي لغة أو مجال.

3 عوامل تحدد توزيع المكوّنات على الأجهزة

العامل	الشرح
Hardware/Software Requirements	المكوّن يحتاج معمارية محددة ← يُنشر على جهاز يدعمها
Availability Requirements	أنظمة High-Availability ← نشر على أكثر من جهاز لضمان الاستمرارية
Component Communications	تواصل مكثّف بين مكوّنات ← أفضل نشرها على نفس الجهاز لتقليل Latency

UML Deployment Diagrams = الطريقة الرسمية لتوثيق توزيع المكوّنات على الأجهزة.

نقاط MCQ احفظها جيداً

- 3 مواضيع في Implementation Issues: **Reuse + Configuration Management + Host-Target**.
- 4 مستويات Reuse: **Abstraction → Object → Component → System**.
- **Abstraction Level** = إعادة استخدام الأفكار (Patterns) لا الكود.
- **Object Level** = مكتبات (Libraries) | **Component Level** = Frameworks | **System Level** = COTS.
- أول خطوة في أي مشروع = التفكير في إعادة الاستخدام قبل التصميم التفصيلي.
- 4 أنشطة CM: **Version Management + System Integration + Problem Tracking + Release Management**.
- **Git** و **Subversion** = أدوات Version Management | **Bugzilla** = Problem Tracking.
- Configuration Management هدفها: التكامل المتحكّم + تتبّع التغييرات + بناء النظام.
- **Host** = جهاز التطوير | **Target** = جهاز التشغيل.
- **Platform** = Hardware + OS + Supporting Software.
- **Simulators** = تُحاكي الأجهزة والبيئة — مفيدة لـ Embedded Systems — مكلفة التطوير.
- **Eclipse** = أشهر General-Purpose IDE يعتمد Plug-in Architecture.
- **UML Deployment Diagrams** = لتوثيق توزيع البرمجيات على الأجهزة.
- لتقليل **Communications Latency** ← نشر المكوّنات المتواصلة على نفس الجهاز أو قريبة منه.

التطوير مفتوح المصدر (Open-Source Development)

ما هو Open-Source Development؟

- **Open-Source Development** = نهج في تطوير البرمجيات يتم فيه **نشر الكود المصدري للعموم** ودعوة المتطوعين للمشاركة في التطوير.
- **جذوره:** نشأ من **Free Software Foundation (FSF)** التي تنادي بأن الكود يجب أن يكون متاحاً للجميع للفحص والتعديل — بدايةً كان الكود يُديره مجموعة صغيرة، ثم جاء Open-Source ليوسّع المشاركة عبر الإنترنت.

---

1 أمثلة على أشهر مشاريع Open-Source

المشروع	المجال
Linux	نظام تشغيل — الأكثر استخداماً في الـ Servers
Apache	Web Server — الأكثر انتشاراً في العالم
Java	لغة برمجة
Eclipse IDE	بيئة تطوير
MySQL	قاعدة بيانات
Android	نظام تشغيل للأجهزة المحمولة

IBM و **Oracle** من أبرز الشركات الكبرى التي تدعم حركة Open-Source وتبني برمجياتها عليها.

---

2 فوائد Open-Source

- **رخيص أو مجاني** — يمكن تحميله دون رسوم في الغالب.
- **موثوقية عالية** — قاعدة مستخدمين كبيرة تكتشف الأخطاء وتُصلحها بسرعة أكبر من البرمجيات المُغلقة.
- **الأخطاء تُكتشف وتُصلح بسرعة أكبر** من Proprietary Software لأن المستخدمين يُصلحون المشاكل بأنفسهم بدلاً من انتظار الشركة.

---

3 السؤالان اللذان تواجههما الشركة

عندما تُقرر شركة استخدام Open-Source تواجه سؤالين:

السؤال	متى يكون الجواب "نعم"؟
هل نستخدم **مكوّنات Open-Source** في منتجنا؟	عند توفر أنظمة Open-Source عالية الجودة في المجال المطلوب ← توفير وقت وتكلفة.
هل نتبنّى **نهج Open-Source** في تطوير برمجياتنا الخاصة؟	عندما يعتمد النموذج التجاري على **بيع الدعم** لا بيع المنتج نفسه.

---

4 متى قد تتجنب الشركة Open-Source؟

- إذا كانت المتطلبات التنظيمية خاصة جداً وغير متوافقة مع الأنظمة المتاحة.
- إذا خشيت الشركة **كشف أسرارها التجارية** للمنافسين.
- إذا كانت أنظمتها الموجودة **غير متوافقة** مع مكوّنات Open-Source المتاحة.

لكن: حتى لو كان التكامل صعباً، قد يكون **تعديل نظام Open-Source** أسرع وأرخص من إعادة بناء الوظيفة من الصفر.

---

5 حقيقة مهمة: Open-Source لا يضمن مشاركة المجتمع

- **نشر الكود** لا يعني بالضرورة **أن المجتمع سيُساهم في تطويره**.
- معظم المشاريع الناجحة كانت **منصات (Platform Products)** لا تطبيقات متخصصة.
- آلاف المشاريع على **Sourceforge** و **GitHub** لديها تنزيلات محدودة جداً.
- **ميزة أخرى:** يطمئن المستخدمون لأنهم يمكنهم الحصول على نسختهم الخاصة لو توقفت الشركة.

---

6 — تراخيص المصدر المفتوح: Open-Source Licensing

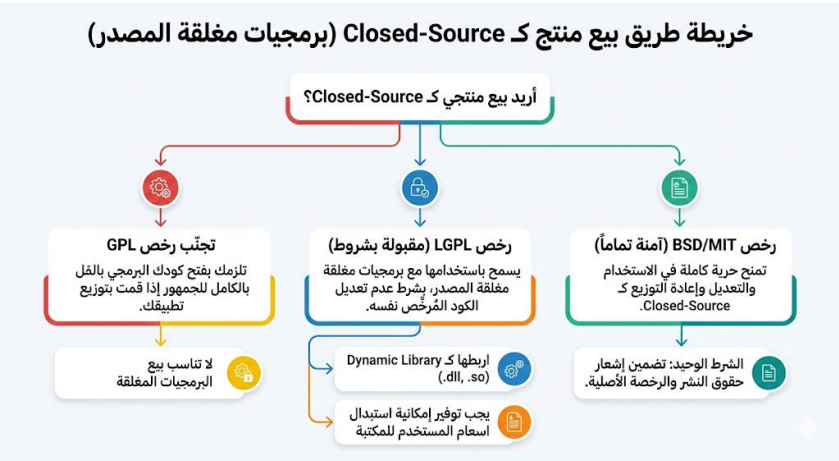
المبدأ الأساسي: مفتوح المصدر ≠ بلا قيود — المطوّر (شخص أو شركة) **يملك الكود قانونياً** ويمكنه وضع شروط ملزمة في الترخيص.

أنواع التراخيص الثلاثة (الأهم للامتحان):

الترخيص	النوع	الشرط الرئيسي	ملاحظة
GNU GPL	Reciprocal (متبادل)	إذا استخدمت كوداً GPL ← **يجب** أن يكون نظامك open-source	الأكثر تقييداً
GNU LGPL	Reciprocal جزئي	يمكنك كتابة مكوّنات تستخدم الكود دون نشرها، لكن **التعديلات على الكود المُرخَّص** يجب نشرها	أقل تقييداً من GPL
BSD License	Non-Reciprocal	لا يلزمك بنشر التعديلات — يمكن دمجه في أنظمة مُغلقة تُباع — **يجب الإشارة للمصدر الأصلي فقط**	الأكثر مرونة

MIT License = نوع من BSD بنفس الشروط تقريباً.

الخلاصة العملية للتراخيص:



6 توصيات Bayersdorfer لإدارة Open-Source في الشركات:

1. أنشئ نظام تتبُّع **للمكوّنات المُنزَّلة** مع نسخة من الترخيص الساري وقت الاستخدام.
2. **اعرف أنواع التراخيص** قبل استخدام أي مكوّن — قد تستخدمه في نظام لكن لا تستخدمه في آخر.
3. **تابع مسار تطور المكوّنات** — افهم كيف قد يتغير المكوّن مستقبلاً.
4. **ثقِّف المطورين** عن Open-Source وشروط التراخيص — لا يكفي وضع إجراءات فقط.
5. **ضع أنظمة تدقيق (Auditing)** — المطورون تحت ضغط المواعيد قد يخرقون شروط الترخيص.
6. **شارك في مجتمع Open-Source** — إذا اعتمدت على منتجاتهم، ادعم تطويرهم.

---

7 نموذج العمل التجاري لـ Open-Source

النموذج: الشركات تنشر الكود مجاناً وتكسب من:

- بيع خدمات الدعم والاستشارات.
- بيع خدمات **Cloud-Based**.
- بيع إصدارات متخصصة لعملاء بعينهم.

هذا النموذج **سيزداد أهمية** مع نمو Open-Source كطريقة لتطوير وتوزيع البرمجيات.

نقاط MCQ احفظها جيداً

- ◈ **Open-Source** = نشر الكود المصدري + دعوة متطوعين للمشاركة.
- ◈ **جذوره** = Free Software Foundation (FSF).
- ◈ **Linux + Apache + Java + Eclipse + MySQL + Android** = أشهر أمثلة Open-Source.
- ◈ **IBM و Oracle** تدعمان حركة Open-Source.
- ◈ Open-Source **رخيص أو مجاني** لكن الدعم والتوثيق قد يكون مدفوعاً.
- ◈ الأخطاء تُكتشف وتُصلح **بسرعة أكبر** مقارنة بـ Proprietary Software.
- ◈ **نشر الكود لا يضمن** مشاركة المجتمع — معظم الناجحة كانت Platform Products.
- ◈ **المطوّر يملك الكود** حتى لو كان Open-Source — يضع شروطاً في الترخيص.
- ◈ **GPL** = Reciprocal — يُلزمك بفتح كودك إذا استخدمته ← **تجنّبه عند بيع Closed-Source**.
- ◈ **LGPL** = Reciprocal جزئي — التعديلات على الكود المُرخَّص فقط يجب نشرها.
- ◈ **BSD / MIT** = Non-Reciprocal — الأكثر مرونة — يمكن دمجه في Closed-Source مع ذكر المصدر.
- ◈ **توصيات Bayersdorfer** = 6 توصيات لإدارة Open-Source في الشركات.
- ◈ نموذج عمل Open-Source يعتمد على **بيع الدعم والخدمات السحابية** لا المنتج نفسه.
- ◈ Open-Source يُطمئن العملاء من **الاعتماد على شركة قد تتوقف** — يمكنهم الاحتفاظ بنسختهم.